

Introduction to Operating Systems

David Marchant

Based on slides by Troels Henriksen

2026-09-21

Incorporating material by Mathias Payer

(<https://github.com/HexHive/OSTEP-slides>).

The purpose of operating systems

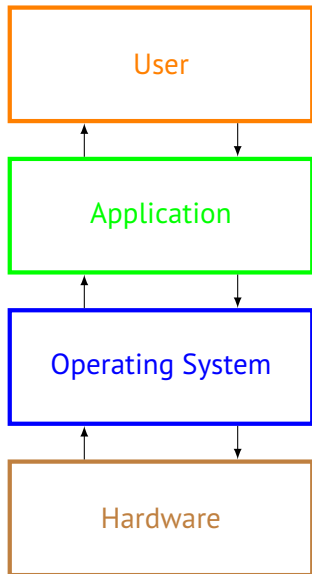
Kernel and Processes

System calls

Process management in Unix

Main takeaways

What is an operating system?



OS is middleware between applications and hardware.

- Provides standardized interface to resources.
- Manages hardware.
- Orchestrates currently executing processes.
- Responds to resource access requests.
- Handles access control.

Why study operating systems?

Inspirational

- One of the most potent *abstractions* in computing.
 - ▶ Each process thinks it has machine to itself.
 - ▶ Controlled communication.
 - ▶ Abstracts over hardware differences.

Practical

- You almost always use an operating system.
- Its performance characteristics are important to understand.
- It often determines what is fundamentally possible.

They are where the magic happens.

In the old days

Each brand of machine would have its own operating system.



IBM System/360 running OS/360 (man not part of computer)



DEC PDP-10 running TENEX

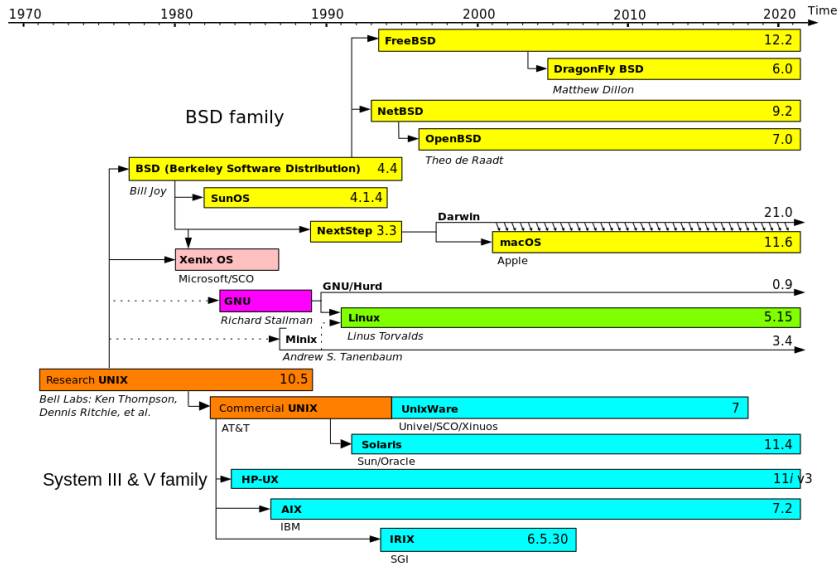


VAX 11/780 running VMS



- PDP-11 running early UNIX, written in C.
- Developed at AT&T, who were banned from selling UNIX.
- Shared it (almost) freely with others, who *ported* UNIX to every machine under the sun.
- Unix was *popular, good enough, and cheap*.

Unix lifetime



What is Unix

What?

- Unix is a family of operating systems derived from original UNIX developed in the 1970s by Ken Thompson and Dennis Ritchie.
- Most modern operating systems are heavily influenced by Unix (even Windows).
- Many operating systems are *direct descendants*: Linux, iOS, macOS, the BSDs, etc.

Why?

- We teach Unix because it is *simple* and *representative* of modern systems.
- We use Unix designs for all examples.
- ...does not mean Unix is always *good* design.

The purpose of operating systems

Kernel and Processes

System calls

Process management in Unix

Main takeaways

More than one process

- So far we've made strictly *sequential* code that does one thing after another until it is done.
- This is a useful (and not inaccurate) concept for how our programs run.
- But is also not entirely true...

More than one process

- So far we've made strictly *sequential* code that does one thing after another until it is done.
- This is a useful (and not inaccurate) concept for how our programs run.
- But is also not entirely true...
- Modern computers run many processes at once, to manage different devices and services.
- If all processes ran entirely sequentially then we'd have to wait for the clock to finish before we could browse the internet
- A key abstraction the OS provides us is the concept of a process, to allow different sequential systems to be interleaved.

Processes contra programs

Program: is a file containing code. Stateless and **dead**.

Process: a running *instance* of a program. The same program can be running in multiple instances. Stateful and **alive**.

They are not the same thing, although we informally often say *program* when we really mean *process*.

Processes contra programs

Program: is a file containing code. Stateless and **dead**.

Process: a running *instance* of a program. The same program can be running in multiple instances. Stateful and **alive**.

They are not the same thing, although we informally often say *program* when we really mean *process*.

- Operating systems manage *processes*.
 - ▶ Switching between multiple *concurrent* processes.
 - ▶ Handling process termination.
 - ▶ Starting new processes (perhaps from a given *program file*).

The kernel

- Technically, *operating systems* encompass lots of parts: shell, GUI, C library, maybe bundled applications (mail reader etc).
- In this course, when we say *operating system* we really mean the **kernel**.

The kernel

- Technically, *operating systems* encompass lots of parts: shell, GUI, C library, maybe bundled applications (mail reader etc).
- In this course, when we say *operating system* we really mean the **kernel**.

Kernel

Always-resident code that services requests from the hardware and manages processes. **It is not a process.**

- The kernel uses the same CPU, memory, and other hardware as ordinary code.
- When running process code, the CPU is in *unprivileged state*, and many operations are restricted (e.g. access to hardware devices).
 - ▶ When an *interrupt* happens, the CPU switches to *privileged state* and jumps to kernel code, which handles it and then resumes the previously running process.
 - ▶ Think of it like a sudden and unplanned procedure call.
 - ▶ Interrupts can be outside events (keyboard press, network traffic) or special instructions (invalid memory accesses, *system calls*).

Virtualising the CPU

Goal Give each process the illusion of exclusive CPU access.

Reality: the CPU is a shared resource.

Virtualising the CPU

Goal Give each process the illusion of exclusive CPU access.

Reality: the CPU is a shared resource.

Solution: **context switching**

- Only one process gets to run at a time.
- ...but we regularly switch between available processes.
- Doing this often and rapidly creates the illusion of simultaneous execution.

Context switching

Intuition Pausing a process, saving its entire *state*, then resuming some other process based on its saved state.

Context switching

Intuition Pausing a process, saving its entire *state*, then resuming some other process based on its saved state.

So what do we need to save?

1. All registers, including control registers.
2. Contents of memory.

Context switching

Intuition Pausing a process, saving its entire *state*, then resuming some other process based on its saved state.

So what do we need to save?

1. All registers, including control registers.
2. Contents of memory.

So when do we do this?

- Regular *timer interrupts* transfer control to the kernel, whose *scheduler* decides the next process to run.
 - ▶ Scheduling is a big and interesting topic that we don't have time to go into.

We've seen this before...

- Recall from our time with Assembler we discussed procedures
- In that context we presented the shift in control as between different function calls
- The same principle applies here, control is shifted from one process to another, though *usually* lacks a direct communication between them.
- But the same principle applies, memory must be saved and maintained between switches.
- But in this context, EVERYTHING must be saved.

The purpose of operating systems

Kernel and Processes

System calls

Process management in Unix

Main takeaways

System calls

- Only the kernel has direct access to hardware and system memory.
- Whenever we want to do IO we have to perform a *system call*.

System calls

A request by a process that the kernel carries out some operation on its behalf.

- Much like a function call, but implemented very differently.

System calls in RISC-V

- The `ecall` (*environment call*) instruction transfers control to the kernel.
 - ▶ Kernel then inspects registers (mostly `a0-a7`) to see what it has been asked to do.
 - ▶ Specific interpretation varies between operating systems.
 - ▶ System call identified by a number.

System calls in RISC-V

- The `ecall` (*environment call*) instruction transfers control to the kernel.
 - ▶ Kernel then inspects registers (mostly `a0-a7`) to see what it has been asked to do.
 - ▶ Specific interpretation varies between operating systems.
 - ▶ System call identified by a number.

In RARS

<https://github.com/TheThirdOne/rars/wiki/Environment-Calls>

- System call number passed in `a7`.
- Excerpt;

Human name	Number	Description	Reads	Writes
<code>PrintInt</code>	1	Prints int to console.	<code>a0</code>	
<code>ReadInt</code>	5	Reads int from console.		<code>a0</code>

How does that work?

- These system calls are both very basic, but also 'higher level' than assembly calls;
- Recall that most input/output really is just file manipulation.
- Many of these OS calls are really just shortcuts to reading or writing to certain files.

Used in A0's io.s (Not any more :P)

```
read_loop:
    beq      t1, t2, read_done
    li       a7, 5
    ecall                                # read int
    sw       a0, 0(t0)
    addi     t0, t0, 4                   # next output addr
    addi     t1, t1, 1                   # increment count
    jal      zero, read_loop
```

System calls in C

- C exposes system calls as functions.
 - ▶ Internally use `ecall` instruction or architecture-specific equivalent.
- Not observably distinct from other C functions, except typically more primitive and tedious to use.

System calls in C

- C exposes system calls as functions.
 - ▶ Internally use `ecall` instruction or architecture-specific equivalent.
- Not observably distinct from other C functions, except typically more primitive and tedious to use.

Example

```
// system calls
int open(const char *pathname, int flags);
ssize_t write(int fd, const void *buf, size_t count);

// stdio functions
FILE *fopen(const char *pathname, const char *mode);
size_t fwrite(const void *ptr, size_t size, size_t nmemb,
              FILE *stream);
```

File descriptors

```
int open(const char *pathname, int flags);
```

```
ssize_t write(int fd, const void *buf, size_t count);
```

- The `int` returned by `open` is a *file descriptor*.
- Has no significance in itself, but allows the kernel to recognise the open file when passed to other system calls.
 - ▶ Typically an index into some kernel-side table.
 - ▶ Such values are known as *handles*.
- Passing complex data structures or pointers between *kernel space* and *user space* is annoying and fragile, so we usually use numeric identifiers instead.

The purpose of operating systems

Kernel and Processes

System calls

Process management in Unix

Main takeaways

Basic principles

- Each process in Unix has a *process ID* (PID).
- Each process has a *parent*.
 - ▶ ...except the *initial process* (`init`) with PID 1.
- A process may have multiple *children*.
- Implies processes are organised as a *tree* (`ps tree` command shows it).
- **Creating processes:** `fork()`.
- **Terminating current process:** `exit()`.
- **Loading program code from disk into current process:** `exec()`.
- **Waiting for a specific child to die:** `waitpid()`.
- **Getting PID of running process:** `getpid()`.

The purpose of operating systems

Kernel and Processes

System calls

Process management in Unix

Main takeaways

Main takeaways

- Hardware provides *mechanisms* such as interrupts, privileged/unprivileged mode, and *virtual memory* (next week).
 - ▶ Kernels implement *policy* and *abstractions* on top.
- Processes are a *purely virtual concept*—CPU has no idea what they are.
- Processes are *isolated* from each other.
- Processes can only directly interact with the outside world through *system calls*, mediated by the kernel.